



CareerHub

Assignment 1.3 — Server State & TanStack Query

A part-by-part walkthrough of how the job page stopped reading a hardcoded array and started fetching live data over HTTP — with a loading skeleton, an error-and-retry state, and a client-side cache. Every part shows the real code, explains the important lines, shows how the UI changes, confirms the requirements, and includes a plain-English explanation for a beginner.

Next.js 15 App Router

React 19

TypeScript (strict)

TanStack Query v5

Tailwind CSS v4

Prepared by Xolisa Matsila · CareerHub Frontend

CONTENTS

What's in this document

Each part follows the same rhythm: the code → line-by-line → why it matters → what the UI does → requirements met → beginner explanation.

Part 1 **Written Decisions — the four conceptual questions**

server vs client state, queryKey, fetch & HTTP errors, SWR

Part 2 **Next.js Route Handler — the mock backend**

src/app/api/jobs/route.ts

Part 3 **Install & Configure TanStack Query**

src/app/providers.tsx + layout.tsx

Part 4 **Environment Variables & API Layer**

.env.local + src/lib/api.ts

Part 5 **Skeleton Loading State**

src/components/JobCardSkeleton.tsx

Part 6 **Replace the Hardcoded Array**

src/app/page.tsx — useQuery

Gate **Build output — zero TS / zero ESLint errors**

npm run build

Q&A **Questions, Answers & Demonstration (added)**

self-test, model answers, and a live data-flow demonstration

HOW TO READ THE GREEN BOXES

Every part ends with a green "Explain it to a beginner" box. If you only read those, you still get the whole story in plain language with no jargon.

The one-sentence summary of Assignment 1.3

We took the list of jobs out of the JavaScript file it was hardcoded in, put it behind a web address (`/api/jobs`), and taught the page to ask for it over the network using a library (TanStack Query) that handles loading, errors, retrying and caching for us — without changing a single one of the visual components from the earlier assignments.

PART 1

Written Decisions

Four conceptual questions answered before writing any code. These explain *why* TanStack Query, not just *how*.

1 SERVER STATE VS CLIENT STATE — FOUR THINGS USEQUERY GIVES YOU FOR FREE

What you get free	If you hand-rolled it with <code>useEffect</code> + <code>useState</code> , and what the user would suffer
Loading & error as first-class values <code>isLoading</code> , <code>isError</code> , <code>error</code>	You'd wire three state variables and reset them in the right order. Miss one path and the user stares at a blank screen with no spinner and no message when the request hangs or fails.
Request deduplication via the <code>queryKey</code>	Ten components needing jobs fire ten identical requests. The browser hammers the API on every screen, wasting bandwidth and slowing every mount.
Caching (stale-while-revalidate)	A manual <code>useEffect([])</code> re-fetches from scratch each mount, so every navigation flashes a spinner for data the user saw seconds ago.
Refetch on focus / reconnect + retries	None of this happens by hand. Leave the tab for an hour → stale listings; one network blip → a permanent error with no self-recovery.

2 THE QUERYKEY CONTRACT

The `queryKey` is the **cache identity** — how the cache stores, looks up, deduplicates and invalidates a result. Same key ⇒ same data; changed key ⇒ a new query and a new fetch.

✗ Accidentally sharing a key: a jobs list and a single job detail both keyed `["jobs"]` overwrite each other — whichever resolves last wins. Symptom: the list page shows one job (or crashes on `.map`), or the detail page renders a whole array of garbage.

✗ Using a unique key when it should be shared: `["jobs", Math.random()]` never matches, so every mount is treated as new. Symptom: a skeleton flashes on every navigation and the cache never warms up — the exact problem TanStack Query was meant to solve.

3 WHY FETCH DOES NOT THROW ON HTTP ERRORS

`fetch` only rejects on a **network failure** (DNS, no connection, CORS, abort). A 404 or 500 is a *successful* round-trip — the server was reached and answered — so the promise resolves and `res.ok` is false (true only for 200–299). Remove the `res.ok` check and TanStack Query sees a resolved `queryFn`, so it stores the error body as success data. The UI leaves loading and renders the success branch against junk — an empty grid, blank fields, or `jobs.map is not a function` crashing the page. The error state never shows. Throwing on `!res.ok` is what routes HTTP errors into the `isError` path.

4 STALE-WHILE-REVALIDATE

Default `staleTime` is `0`, so returning to the tab triggers a background refetch — but the data already on screen stays rendered. The user sees the existing grid, not a spinner; only the background `isFetching` flag flips, and

new data swaps in seamlessly. A `useEffect(..., [])` fetch runs once on mount and never again, so it never refreshes on focus — stale data forever until a full reload. SWR gives freshness without the flash.

EXPLAIN IT TO A BEGINNER

Think of TanStack Query as a smart assistant for fetching data. Instead of you personally writing sticky-notes for "am I still loading?", "did it fail?", "did I already fetch this?", the assistant tracks all of that. The queryKey is just the **LABEL ON THE DRAWER** where it files each result — same label means "you already have this, here it is instantly"; a different label means "go fetch a fresh one."

PART 2

Next.js Route Handler — the mock backend

A tiny server endpoint that answers `GET /api/jobs` with the seed listings as JSON. This is the seam a real production API will slot into later.

● `src/app/api/jobs/route.ts`

```
1 import { NextResponse } from "next/server";
2 import { JOBS } from "@lib/seed-jobs";
3 import type { JobListing } from "@types";
4
5 /** GET /api/jobs - returns the seed listings as JSON. GET only. */
6 export async function GET() {
7   const jobs: JobListing[] = JOBS;
8   return NextResponse.json(jobs);
9 }
```

- L1** `NextResponse` is Next.js's helper for building HTTP responses. `NextResponse.json()` serialises an object and sets the `Content-Type: application/json` header for us.
- L2** We import the `JOBS` array from `seed-jobs.ts` — the same seed data from Assignment 1.1. The component tree no longer imports this directly; only the server does.
- L3** `type` -only import of `JobListing`. Typing the variable means if the seed data ever drifts from the contract, the build fails here — not in the user's browser.
- L6** The key line. Exporting a function literally named `GET` is how the App Router maps this file to the GET method of the `/api/jobs` URL. Because no POST/PUT is exported, any other method automatically gets a **405 Method Not Allowed**.
- L7** Annotating `jobs: JobListing[]` asserts the seed matches the contract the frontend expects.
- L8** Return the JSON response. This is the entire backend for now — a real database call would replace this one line without touching anything else.

WHY THIS MATTERS

Before this file, the jobs lived inside a React file and could only change by editing code and redeploying. Now they live behind a **WEB ADDRESS**. The front-end fetches them like it would from any real server, which means the day a real API exists, we change one URL — not the UI.

EFFECT ON THE UI

Directly, none — this is a server endpoint with no visual output of its own. Indirectly, it is the data source the entire job grid now renders from. You can prove it by opening `http://localhost:3000/api/jobs` in the browser and seeing the raw JSON array — every field of `JobListing` present, all four employment types represented, and the Luno DevOps listing with `"isActive": false`.

- ✓ Route handler at `src/app/api/jobs/route.ts`
- ✓ Returns the Assignment 1.1 seed listings via `NextResponse.json()`
- ✓ Responds to GET only (no other method exported ⇒ 405)
- ✓ Data covers all four `employmentType` values — FullTime, PartTime, Contract, Internship
- ✓ Includes at least one `isActive: false` listing (Luno DevOps)

EXPLAIN IT TO A BEGINNER

Imagine a vending machine. Before, the snacks were glued inside the app's code. Now we built a little hatch with a label that says **"GET /API/JOB"**. When anyone knocks with a GET request, the hatch hands back the list of jobs as JSON (a universal text format for data). Knock with any other kind of request and the hatch stays shut (error 405). That hatch is a "route handler."

Install & Configure TanStack Query

Create the provider that gives every component access to the query cache, and wire it into the root layout — keeping the client boundary in exactly one file.

● installed once

```
npm install @tanstack/react-query @tanstack/react-query-devtools
```

● src/app/providers.tsx

```
1  "use client";
2
3  import { useState, type ReactNode } from "react";
4  import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
5  import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
6
7  export default function Providers({ children }: { children: ReactNode }) {
8    const [queryClient] = useState(() => new QueryClient());
9
10   return (
11     <QueryClientProvider client={queryClient}>
12       {children}
13       <ReactQueryDevtools initialIsOpen={false} />
14     </QueryClientProvider>
15   );
16 }
```

L1 "use client" marks this as a Client Component. TanStack Query uses React state and the browser, so its provider must run on the client. Keeping this directive here means the layout can stay a Server Component.

L7 The component accepts and forwards a `children: ReactNode` prop — it wraps whatever the app renders.

L8 The most important line. The `QueryClient` is created with the `useState` initialiser form — `useState(() => new QueryClient())`. The function runs once per mount and never again on re-render, so each browser session gets exactly one client.

L11 `QueryClientProvider` puts that client into React context so any `useQuery` deep in the tree can reach the same cache.

L13 `ReactQueryDevtools` with `initialIsOpen={false}` mounts the floating debug panel, collapsed by default.

WHY NOT A MODULE-LEVEL `CONST QUERYCLIENT = NEW QUERYCLIENT()`?

A module-level client is created **ONCE ON THE SERVER** and shared across every incoming request. One user's cached jobs (or, in a real app, their private data) could leak into another user's response. The

`useState` initialiser scopes the client to a single browser session and creates it exactly once — correct on both server and client.

- src/app/layout.tsx – the wiring (Server Component, no "use client")

```
import Providers from "../providers";
// ...
export default function RootLayout({ children }) {
  return (
    <html><body>
      <Providers>                                // ← client boundary lives inside Providers
        <AuthProvider><ToastProvider>
          <Navbar />
          <main>{children}</main>
          <Footer />
        </ToastProvider></AuthProvider>
      </Providers>
    </body></html>
  );
}
```

EFFECT ON THE UI

A small **TANSTACK QUERY LOGO BUTTON APPEARS IN THE BOTTOM-LEFT CORNER** of every page. Click it and the DevTools panel slides open, listing live cache entries. Right after install it is empty — no query has run yet. (Everything else on the page looks identical; the provider is plumbing, not pixels.)

CareerHub

Menu

...page content...



← TanStack Query Devtools button

✓ Requirements met

- ✓ Both packages installed
- ✓ `providers.tsx` is a Client Component (`"use client"`)
- ✓ `QueryClient` created with the `useState` initialiser form, not a module-level variable
- ✓ `children` wrapped in `QueryClientProvider` ; `children: ReactNode` accepted & forwarded
- ✓ `ReactQueryDevtools` mounted with `initialIsOpen={false}`
- ✓ `layout.tsx` wraps `{children}` in `<Providers>` and gains no `"use client"`

EXPLAIN IT TO A BEGINNER

Providers is like installing the **WATER MAINS** for a building before anyone turns on a tap. No one sees the pipes, but now every room (component) can connect to the same water supply (the query cache). We build

the mains once per visit (that's the `useState` trick) so we don't accidentally re-plumb the whole building on every little change — and we put one tap in the basement (the DevTools button) to inspect the water.

PART 4

Environment Variables & API Layer

A configurable base URL kept out of version control, and one pure function that performs the HTTP call — with no React anywhere near it.

● `.env.local` (git-ignored)

```
NEXT_PUBLIC_API_URL=http://localhost:3000
```

`.env*` is listed in `.gitignore`, so this file never gets committed. The `NEXT_PUBLIC_` prefix is what makes the value readable in browser code.

● `src/lib/api.ts`

```
1 import type { JobListing } from "@types";
2
3 export async function fetchJobs(): Promise<JobListing[]> {
4   const baseUrl = process.env.NEXT_PUBLIC_API_URL ?? "";
5   const res = await fetch(`${baseUrl}/api/jobs`);
6
7   if (!res.ok) {
8     throw new Error(
9       `Failed to load jobs – server responded with ${res.status} ${res.statusText}`
10    );
11  }
12
13  return (await res.json()) as JobListing[];
14 }
```

L1 Only a type import — no React, no components. This keeps `fetchJobs` pure network logic that can be unit-tested or reused anywhere.

L4 Read the base URL from the environment. The `?? ""` fallback means that if the env var is missing, we use a relative `/api/jobs` — the app still works against its own backend instead of fetching `undefined/api/jobs` and 404-ing.

L5 Build the full URL and `await fetch`. This is the actual network request.

L7–11 The crucial check. `fetch` does not throw on 404/500, so we test `res.ok` ourselves and throw a descriptive Error that includes the status code. This is what pushes HTTP failures into TanStack Query's `isError` path (see Part 1, Q3).

L13 Parse the JSON and assert it as `JobListing[]` — the typed value `useQuery` will hand to the components as `jobs`.

WHY A SEPARATE FILE WITH NO REACT

Keeping the network call out of the component means it has one job and can be tested in isolation. The component asks "give me jobs"; how that happens (which URL, error handling) is this file's concern alone. Swap the data source later and the components never notice.

✓ Requirements met

- ✓ `.env.local` with `NEXT_PUBLIC_API_URL`, git-ignored via `.env*` in `.gitignore`
- ✓ `src/lib/api.ts` exports a single function `fetchJobs`
- ✓ Reads the base URL from `process.env.NEXT_PUBLIC_API_URL` and builds the full URL
- ✓ Calls `fetch`, checks `res.ok`, throws a descriptive Error including the HTTP status code
- ✓ Returns parsed JSON typed as `JobListing[]`; imports nothing from React

EXPLAIN IT TO A BEGINNER

`fetchJobs` is a **COURIER**. You give it nothing; it goes to the address written on the env-var sticky note, knocks (fetch), and comes back with the jobs. If the address answers with "Not Found" or "Server Error," the courier doesn't quietly hand you an empty box — it **RAISES ITS HAND AND SAYS EXACTLY WHAT WENT WRONG** (the thrown Error with the status code), so the app can show a proper "try again" screen instead of pretending it worked.

Skeleton Loading State

A grey placeholder that mirrors the real card region-for-region, so the page reserves the right shape while data loads — and never jumps when it arrives.

● src/components/JobCardSkeleton.tsx (abridged)

```
1  /** A single pulsing placeholder bar. */
2  function Bar({ className }: { className?: string }) {
3      return <div className={`animate-pulse rounded bg-slate-200 dark:bg-slate-700
4      ${className}`} />;
5  }
6
7  export function JobCardSkeleton() {
8      return (
9          <div aria-hidden="true" className="flex h-full flex-col rounded-2xl border ... p-5
10         ...">
11              <div className="flex items-start justify-between"> { /* badge + bookmark */}
12                  <Bar className="h-5 w-20" /> <Bar className="h-6 w-6 rounded-lg" />
13              </div>
14              <Bar className="mt-4 h-5 w-3/4" /> { /* title */}
15              <Bar className="mt-2 h-4 w-1/2" /> { /* company · location */}
16              <Bar className="mt-4 h-4 w-2/5" /> { /* salary */}
17              <Bar className="mt-2 h-3 w-1/3" /> { /* closing date */}
18              <div className="mt-auto flex gap-3 pt-6"> { /* footer */}
19                  <Bar className="h-3 w-24" /> <Bar className="h-3 w-16" />
20              </div>
21          </div>
22      );
23  }
24
25  export function JobListSkeleton() {
26      return (
27          <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
28              {Array.from({ length: 6 }).map((_, i) => <JobCardSkeleton key={i} />)}
29          </div>
30      );
31  }
```

L2–4 `Bar` is one pulsing grey block. `animate-pulse` gives the breathing effect; `bg-slate-200` + `dark:bg-slate-700` mean it reads correctly in both light and dark mode.

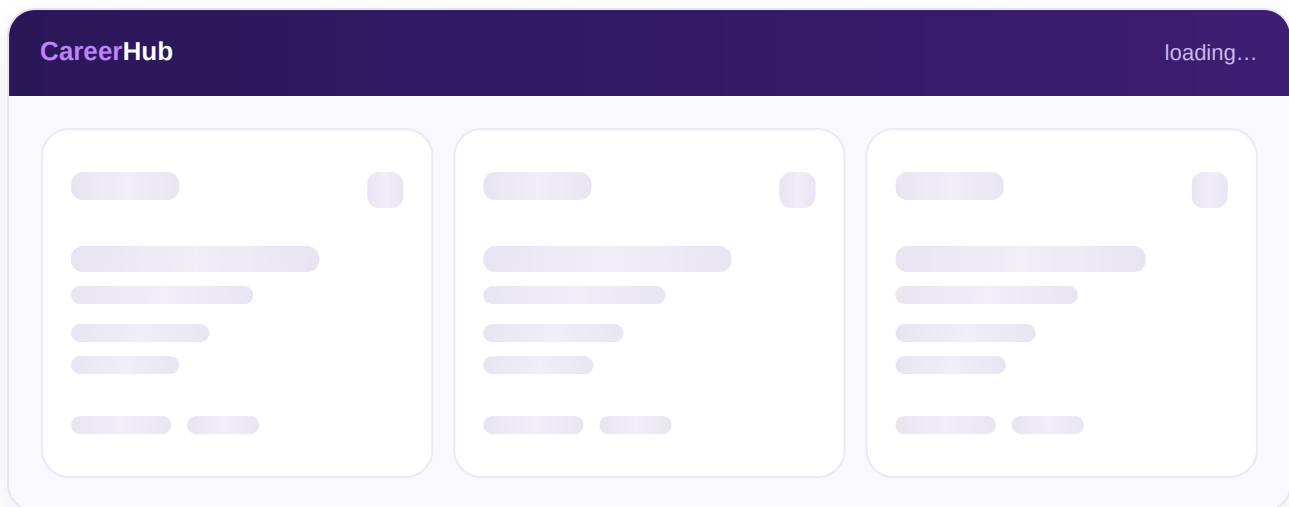
L8 `aria-hidden="true"` hides the decorative skeleton from screen readers — there's no real content to announce.

L9–18 Every region `JobCard` renders has a matching bar: badge row, title, company line, salary, closing date, and a footer pinned with `mt-auto`. No real text — never "Loading...".

L25–26 `JobListSkeleton` renders six cards in the exact grid `JobList` uses (`grid-cols-1 sm:grid-cols-2 lg:grid-cols-3`). Six matches a realistic first page and keeps the loading frame the same height as the loaded one.

EFFECT ON THE UI — WHAT YOU ACTUALLY SEE WHILE LOADING

Six card-shaped grey placeholders, pulsing, laid out in the same 1/2/3-column grid the real jobs use. Throttle the network to Slow 3G and they hold for several seconds before the real grid fades in — with **NO LAYOUT JUMP**, because they occupy the identical footprint.



✓ Requirements met

- ✓ `JobCardSkeleton` mirrors every region of `JobCard` (badge, title, company, salary, closing line, footer)
- ✓ Placeholders use `animate-pulse` with both `bg-slate-200` light () and `dark:bg-slate-700` dark () backgrounds
- ✓ No real text — no "Loading..." content
- ✓ `JobListSkeleton` renders six cards in the same grid layout as `JobList`

EXPLAIN IT TO A BEGINNER

A skeleton is like the **PENCIL OUTLINE AN ARTIST SKETCHES BEFORE PAINTING**. While the real picture (the job data) is still on its way, we draw grey boxes exactly where the title, company and salary will go. Because the outline is the same size as the finished card, when the colour fills in nothing slides around. A plain spinner can't promise that — it's tiny, so the page lurches when six full cards suddenly appear.

Replace the Hardcoded Array

The payoff. `page.tsx` deletes its hardcoded data and asks `useQuery` for jobs instead — then renders one of three states: skeleton, error, or grid.

● `src/app/page.tsx` – the query (the rest of the file is unchanged)

```
6 import { useQuery } from "@tanstack/react-query";
8 import { JobListSkeleton } from "@components/JobCardSkeleton";
11 import { fetchJobs } from "@lib/api";
...
32 const {
33   data: jobs,    // ← renamed from `data` to `jobs` on destructure
34   isPending,
35   isError,
36   error,
37   refetch,
38 } = useQuery({
39   queryKey: ["jobs"],
40   queryFn: fetchJobs,
41 });
```

L32–37 Destructure the five values we need from `useQuery`'s return. `data` is renamed to `jobs` so the rest of the file reads naturally.

L39 `queryKey: ["jobs"]` — the cache identity. Any component asking for `["jobs"]` shares this one fetch and its cached result.

L40 `queryFn: fetchJobs` — the courier from Part 4. TanStack Query calls it, tracks its loading/error, and caches what it returns.

● `src/app/page.tsx` – the three mutually-exclusive states

```
192 {isPending ? (
193   /* PENDING – only the skeleton grid renders */
194   <JobListSkeleton />
195 ) : isError ? (
196   /* ERROR – show the real message + a retry button */
197   <div role="alert" className="... border-red-200 bg-red-50 dark:border-red-900/50 ..." >
203     <h3>We couldn't load the job listings</h3>
207     <p>{error.message}</p>
211     <button onClick={() => refetch()}>Try again</button>
216   </div>
217 ) : (
218   /* SUCCESS – `jobs` is defined; render the real grid */
224   <JobList jobs={visibleJobs} ... />
225 )}
```

192 Pending: render `<JobListSkeleton />` and nothing else — the six pulsing cards from Part 5.

196–216 **Error:** a styled panel (with light and dark classes) showing the actual `error.message`, plus a "Try again" button that calls `refetch()`. The job grid is not rendered.

217 **Success:** `jobs` is guaranteed defined here, so the filter bar, optional summary panel and `<JobList />` render against live data — passing `jobs` as the prop `JobList` already expects.

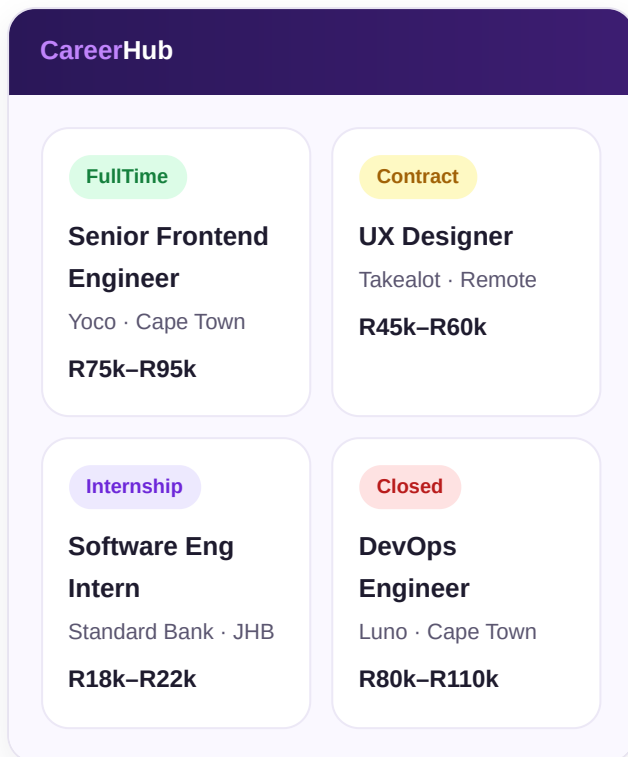
THE SESSIONSTORAGE ADAPTATION

The 1.2 restore effect validated the stored id with `jobs.some(j) => j.id === stored`. On mount `jobs` is now `undefined` (the query is pending), so that guard could never pass. It is **REMOVED**: we restore the id unconditionally. If it no longer matches any loaded job, `selectedJob` simply resolves to `null` and the summary panel renders nothing — graceful degradation, no crash.

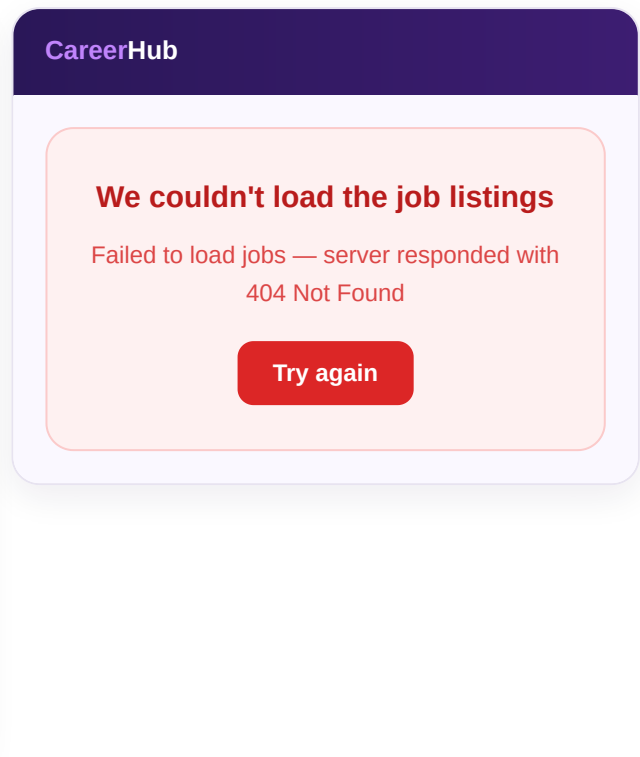
EFFECT ON THE UI — THREE STATES, ONE PAGE

On first load you see the **SKELETON**; a moment later the **REAL JOB GRID**. Point the env var at a dead port and you get the **RED ERROR PANEL** with a working "Try again" button. Select a job and refresh — it stays selected once data loads. The components rendering all this (JobCard, JobList) are **BYTE-FOR-BYTE UNCHANGED** from 1.2.

SUCCESS state — live data



ERROR state — retry



```
mount → useQuery runs fetchJobs → isPending → skeleton → resolves → jobs → grid
      \ rejects → isError → panel + retry
```

✓ Requirements met

✓ Hardcoded jobs array removed; replaced with a `useQuery` call

✓ `queryKey: ["jobs"]` , `queryFn: fetchJobs`

✓ Deconstructs `data` (renamed `jobs`), `isPending` , `isError` , `error` , `refetch`

✓ Pending → `<JobListSkeleton />` only; `error.message` + "Try again" `refetch()` ; `<JobList />`
→ `Error` → styled panel + `Success` →

✓ Error panel has light and dark classes; guard prevents rendering JobList while `jobs` is undefined

✓ sessionStorage validation removed — graceful degradation when a stored id no longer matches

✓ `JobCard` , `JobList` , `JobStatusBadge` , `ThemeToggle` , `AvailabilityBadge` not modified

EXPLAIN IT TO A BEGINNER

This is the moment the page stops *knowing* the jobs and starts *asking* for them. `useQuery` is the page raising its hand and saying "I need the 'jobs' list." While it waits, it shows the pencil-outline skeleton. If the answer comes back, it shows the real cards. If something breaks, it shows a polite "try again" card. Three outcomes, all handled — and the cards themselves never had to change, because they only ever cared about receiving a list, not where it came from.

Build output — zero errors

`npm run build` completes with zero TypeScript errors and zero ESLint errors. The new `f` `/api/jobs` route is server-rendered on demand.

```
● npm run build

> careerhub-frontend@0.1.0 build
> next build

▲ Next.js 15.5.19
- Environments: .env.local

Creating an optimized production build ...
✓ Compiled successfully in 59s
Linting and checking validity of types ...
Collecting page data ...
✓ Generating static pages (13/13)
Finalizing page optimization ...

Route (app)                Size      First Load JS
┌ ○ /                      22.7 kB   134 kB
├ ○ /_not-found            993 B     103 kB
├ ○ /about                 810 B     107 kB
├ f /api/jobs              123 B     103 kB
├ ○ /applications         5.95 kB   112 kB
├ f /apply/[jobId]        7.18 kB   113 kB
├ ○ /contact              797 B     103 kB
├ ○ /forgot-password      2.59 kB   108 kB
├ ○ /login                3.07 kB   109 kB
├ ○ /recruiter            5.66 kB   112 kB
├ ○ /recruiter/signin     5.1 kB    108 kB
└ ○ /signup               3.16 kB   109 kB
+ First Load JS shared by all      102 kB

○ (Static)   prerendered as static content
f (Dynamic)  server-rendered on demand
```

🚩 THE WHOLE ASSIGNMENT IN ONE BREATH

A route handler (Part 2) serves jobs over HTTP. A provider (Part 3) gives the app a query cache. A pure API function (Part 4) fetches and validates the response. A skeleton (Part 5) holds the layout while loading. And `page.tsx` (Part 6) ties it together with one `useQuery` call that renders a skeleton, an error-with-retry, or the live grid — all without touching a single visual component from the earlier assignments.

Questions, Answers & Demonstration

A self-test on the concepts behind this assignment, with model answers, followed by a worked demonstration of the data flowing from URL to grid. Useful for revision and for interview prep.

PART A — SELF-TEST QUESTIONS (NO ANSWERS)

#	Question
Q1	Name four things <code>useQuery</code> gives you for free that you would otherwise hand-wire with <code>useEffect</code> + <code>useState</code> .
Q2	What is a <code>queryKey</code> , and what breaks if two unrelated queries accidentally share one?
Q3	Why does <code>fetch</code> not throw on a 404 or 500, and what must you write so those land in the <code>isError</code> path?
Q4	Why is the <code>QueryClient</code> created with <code>useState(() => new QueryClient())</code> instead of a module-level <code>const</code> ?
Q5	The skeleton renders exactly six cards in the same grid as <code>JobList</code> . Why six, and why match the grid?
Q6	What does the <code>NEXT_PUBLIC_</code> prefix do, and why is <code>.env.local</code> git-ignored?
Q7	The 1.2 <code>sessionStorage</code> restore used <code>jobs.some(j => j.id === stored)</code> . Why did that guard have to be removed in 1.3, and what replaces its safety?
Q8	Which visual components changed in this assignment, and what does that tell you about the architecture?

PART B — MODEL ANSWERS

A1. Loading & error as first-class values (`isPending`, `isError`, `error`); request deduplication via the `queryKey`; caching with stale-while-revalidate; and automatic refetch on focus/reconnect plus retries. By hand you would juggle three state variables, fire duplicate requests, re-fetch on every mount, and never self-recover from a blip.

A2. The `queryKey` is the cache identity — how a result is stored, looked up, deduplicated and invalidated. If a list and a detail both use `["jobs"]`, whichever resolves last overwrites the other: the list may render one job or crash on `.map`, or the detail page renders an array.

A3. `fetch` only rejects on a network failure (DNS, no connection, CORS, abort). A 404/500 is a completed round-trip, so the promise resolves with `res.ok === false`. You must check `res.ok` and `throw` yourself; otherwise the error body is stored as success data and the success branch renders against junk.

A4. A module-level client is created once on the server and shared across every request, so one user's cached data could leak into another's response. The `useState` initialiser runs once per mount and scopes the client to a single browser session — correct on both server and client.

A5. Six matches a realistic first page of results, and rendering the skeleton in the identical `grid-cols-1 sm:grid-cols-2 lg:grid-cols-3` layout keeps the loading frame the same height as the loaded one — so the content never jumps when real data arrives.

A6. `NEXT_PUBLIC_` exposes the variable to browser-side code (without it the value is server-only). `.env.local` is git-ignored via `.env*` so environment-specific URLs and any secrets never get committed.

A7. On mount, `jobs` is `undefined` while the query is pending, so `jobs.some(...)` could never pass (and would risk a crash). The id is now restored unconditionally; if it matches nothing, `selectedJob` resolves to `null` and the summary panel simply renders nothing — graceful degradation instead of a guard.

A8. None of the visual components (`JobCard` , `JobList` , `JobStatusBadge` , `ThemeToggle` , `AvailabilityBadge`) changed. That proves the data source was swapped behind a clean seam — the UI only ever depended on receiving a typed list, not on where it came from.

DEMONSTRATION

Walk the request, end to end

A concrete trace of one page load — every hop the data takes from the browser knocking on the URL to the grid painting on screen — plus the two failure detours.

THE HAPPY PATH, STEP BY STEP

1. Page mounts → `useQuery({ queryKey: ["jobs"], queryFn: fetchJobs })` runs
2. Cache has no entry for `["jobs"]` → `isPending = true` → `<JobListSkeleton />` renders
3. `fetchJobs()` builds `http://localhost:3000/api/jobs` and awaits fetch
4. Route handler `GET()` returns `NextResponse.json(JOBS)` → 200 OK
5. `res.ok === true` → JSON parsed → typed as `JobListing[]`
6. Promise resolves → cache stores result under `["jobs"]` → `isPending = false`
7. Success branch renders → `<JobList jobs={visibleJobs} />` → grid paints, no layout jump

TRY IT YOURSELF (COMMANDS)

- terminal – prove each piece in isolation

1. Start the dev server

```
npm run dev
```

2. Hit the route handler directly – should print the JSON array

```
curl http://localhost:3000/api/jobs
```

3. Prove GET-only – any other method should give 405

```
curl -X POST http://localhost:3000/api/jobs -i | head -n 1
```

→ HTTP/1.1 405 Method Not Allowed

4. Force the error state – point the env var at a dead port, restart, reload

.env.local: NEXT_PUBLIC_API_URL=http://localhost:9999

→ red panel: "Failed to load jobs – server responded with ..."

THE TWO FAILURE DETOURS

If this happens...	What the user sees
Network failure (server down, wrong port) → <code>fetch</code> rejects	<code>isError = true</code> → red panel with the thrown message and a working "Try again" button calling <code>refetch()</code> .
HTTP error (404/500) → <code>fetch</code> resolves but <code>res.ok</code> is false	The <code>!res.ok</code> guard throws a descriptive <code>Error</code> with the status code → same red panel, message includes e.g. <i>404 Not Found</i> .
Stale tab refocus → background refetch	Existing grid stays on screen; only <code>isFetching</code> flips; fresh data swaps in with no spinner flash.

WHAT "DONE" LOOKS LIKE

✓ Acceptance demonstration

- ✓ First load shows the pulsing six-card skeleton, then the live grid — no layout jump.
- ✓ `/api/jobs` returns valid JSON to `curl` ; POST returns 405.
- ✓ Pointing the env var at a dead port shows the red error panel; "Try again" recovers once the server is back.
- ✓ Selecting a job then refreshing keeps it selected once data loads (graceful when the id is gone).
- ✓ `npm run build` → zero TypeScript errors, zero ESLint errors; `npm run dev` server-rendered on demand.

EXPLAIN IT TO A BEGINNER

Think of it as ordering food. The page places an order ("I need the jobs"), and while the kitchen cooks it shows you a picture of the empty plate the right size (the skeleton). If the food arrives, it's plated exactly where the picture was (no jump). If the kitchen is closed or the order's wrong, you get a clear note explaining why and a button to order again — never a blank table and never a wrong dish served as if it were right.